

Solution to analysis in Home Assignment 4

Shiming Long
shimingl@chalmers.se

May 17, 2026

1 Smoothing

(a)

Based on the Coordinated Turn Motion Model and dual-bearing sensor measurement data, an Extended Kalman Filter (EKF) and an RTS Smoother were executed respectively. Figure 1 shows the comparison among the true trajectory, measured positions, filtered position trajectory, and smoothed position trajectory.

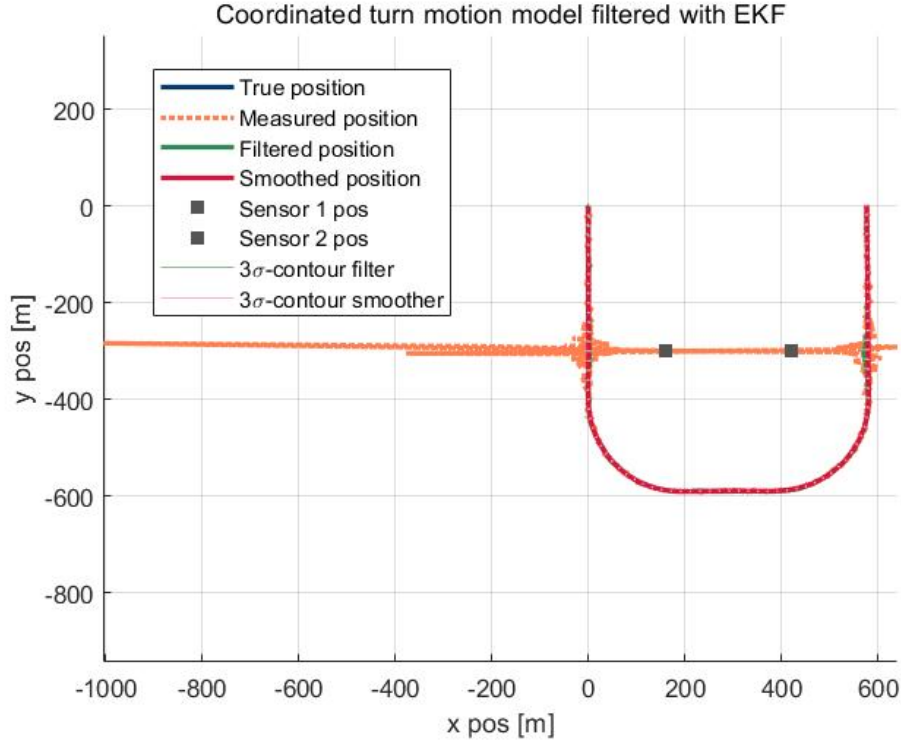


Figure 1: Position Trajectories

In straight sections, both the EKF and smoother effectively suppress measurement noise from the distant sensors (near $y = -300$) and track the true trajectory. However, their performance diverges during the 90° turns. The EKF exhibits lag and overshoot because it is a causal system. Relying only on past and present measurements ($y_{1:k}$), its inertial prediction continues straight until new data forces a correction. Conversely, the RTS smoother acts as a non-causal system. By utilizing the complete measurement sequence ($y_{1:K}$) during its backward recursion, it incorporates future data to "foresee" turns, entirely eliminating overshoot and delay.

Figure 2 illustrates the evolution of three state components: Velocity, Heading, and Turn rate over time.

During turns, the true turn rate forms an ideal square wave. The EKF struggles with these step changes, exhibiting noticeable delays and high-frequency oscillations caused by angle measurement noise. Conversely, the RTS smoother incorporates global contextual information during its backward pass to effectively suppress this noise. Consequently, the smoothed state curves remain stable, closely approximating the true square-wave profile.

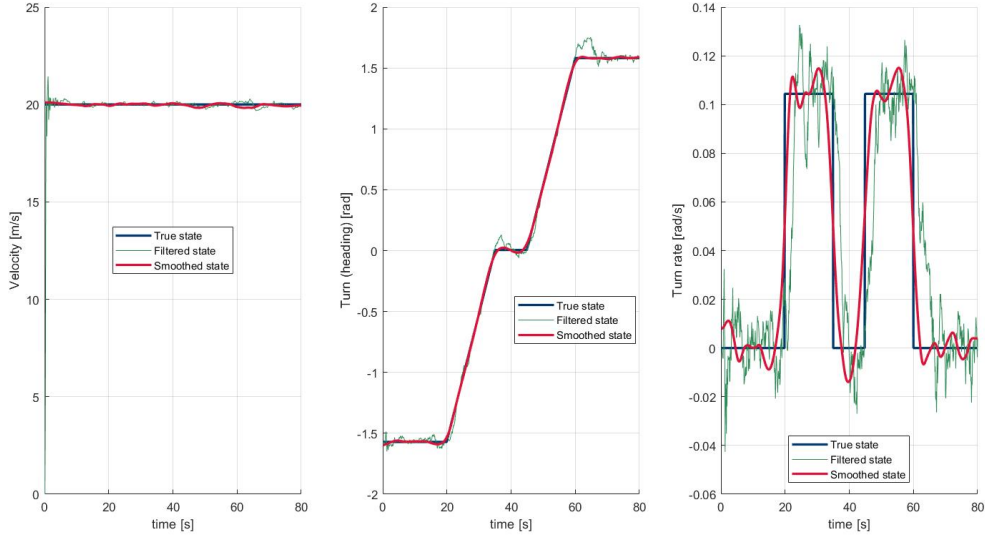


Figure 2: State Trajectories

(b)

At time $k = 400$, a strong impulse-type outlier was manually introduced into the measurement sequence.

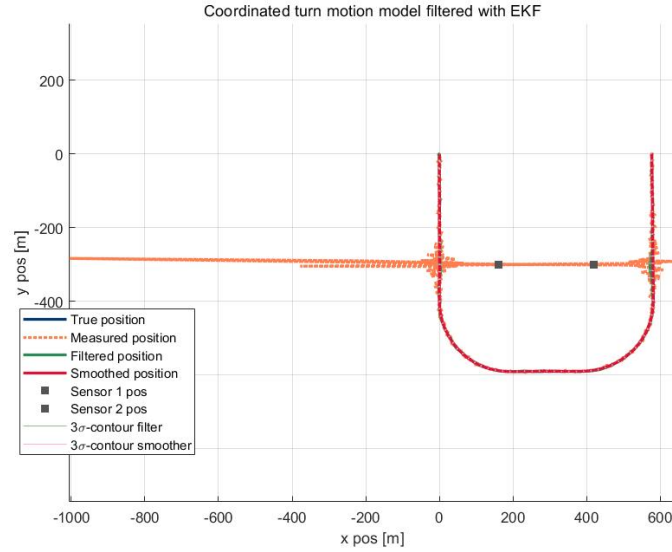


Figure 3: $k=400$ position trajectory

Figure 4 highlights the stark contrast in handling the measurement outlier at $t = 40$ s.

The causal EKF exhibits a massive spike in its heading and turn rate curves. Lacking access to future data, it is forced to trust the anomalous measurement at $k = 400$, misinterpreting the outlier as an instantaneous and violent turning maneuver.

Conversely, the non-causal RTS smoother maintains perfect continuity. During its backward recursion, it incorporates future measurements ($k > 400$) to evaluate the anomaly against its dynamics model. It determines that recovering from such a se-

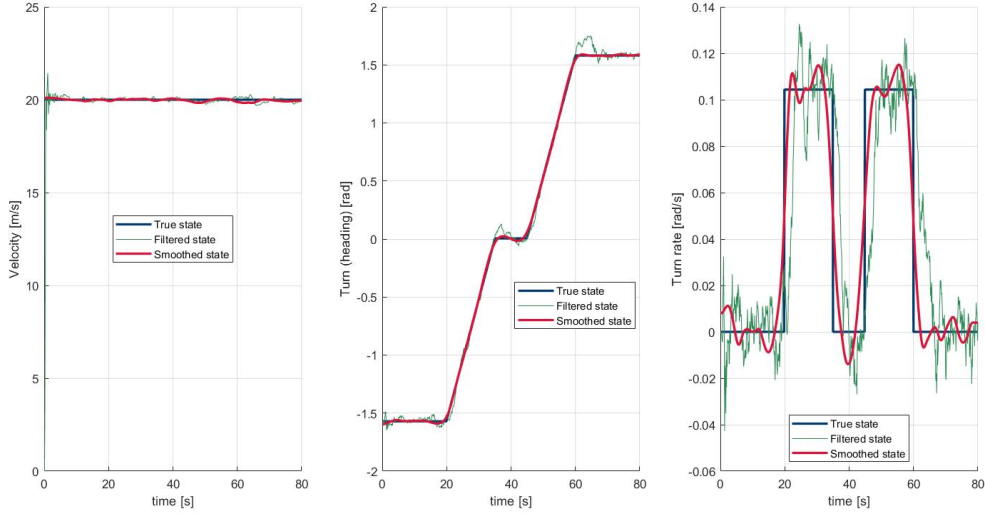


Figure 4: $k=400$ state trajectory

vere yaw to resume a straight path by $k = 401$ is physically impossible, allowing it to successfully identify and reject the outlier.

2 Particle filters for linear and Gaussian systems

(a)

For a linear and Gaussian state-space model, the Kalman Filter (KF) provides the mathematically optimal exact solution. In this task, we evaluate the performance of two Particle Filters (PF)—one with resampling and one without—against this optimal KF baseline.

By evaluating the Root Mean Square Error (RMSE), the KF yields the theoretical minimum error, calculated at 0.8402. To achieve a comparable performance with the PF, empirical tuning demonstrates that approximately $N = 1000$ particles are required. With this configuration, the PF with resampling achieves an RMSE of 0.8568, almost perfectly replicating the tracking precision and uncertainty bounds of the optimal KF. Conversely, the PF without resampling exhibits a significantly higher RMSE of 1.5348, demonstrating its failure to reliably track the true state over extended periods without a resampling mechanism.

Figure 6 illustrates the posterior density distributions at the beginning ($k = 1$), middle ($k = 15$), and end ($k = 30$) of the sequence. At $k = 1$, both particle filters successfully approximate the Gaussian bell curve of the KF. However, by $k = 15$ and $k = 30$, a critical failure occurs in the PF without resampling, known as weight degeneracy. A single particle accumulates a normalized weight approaching 1, while the weights of all other particles drop essentially to 0. The stem plot collapses, meaning the filter loses its ability to represent the probability distribution and track the state.

The PF with resampling effectively solves this degeneracy issue. By periodically duplicating particles in high-likelihood regions and discarding those with negligible weights, the filter continuously populates the state space with relevant samples. As shown in the histogram subplots for $k = 15$ and $k = 30$, the resampled PF maintains a robust particle distribution that consistently and accurately approximates the true Gaussian posterior

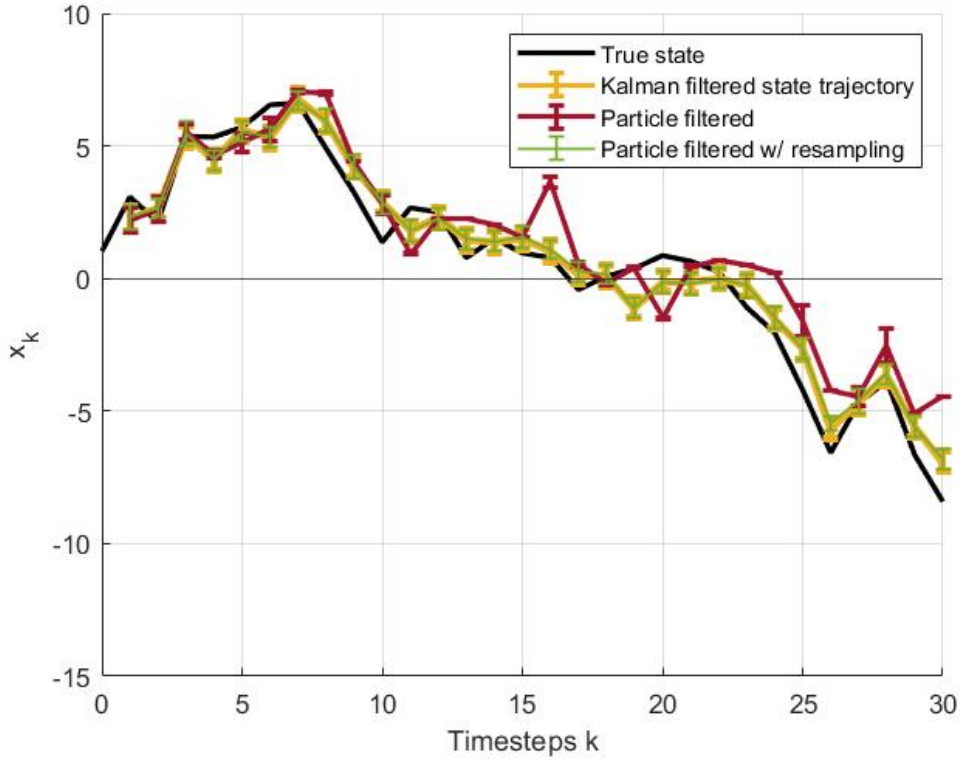


Figure 5: trajectories and errorbars

maintained by the KF.

(b)

In this task, the filters are initialized with an intentionally incorrect prior distribution of $p(x_0) = \mathcal{N}(-20, 2)$, introducing a massive initial deviation from the true starting state, which is near $x = 2$. The objective is to observe how rapidly each filter can compensate for this error and "catch up" to the true trajectory.

The KF rapidly corrects the massive initial offset. Driven by the Kalman gain, it heavily weights incoming correct measurements over the erroneous prior, snapping back to the true trajectory within just a few time steps.

The resampled PF also successfully recovers through stochastic propagation. Process noise diffuses the distant particle cloud, allowing a few particles to drift closer to the true measurements. Resampling then aggressively duplicates these high-weight particles and discards the rest, migrating the entire cloud to the true state.

Conversely, the un-resampled PF fails completely. Since the actual measurements fall deep into the near-zero tails of the distant particles' likelihood functions, severe weight degeneracy occurs immediately. Lacking a resampling mechanism to relocate particles to high-likelihood regions, the filter permanently diverges.

(c)

To visually understand the internal mechanics of the particle filters, the catch-up scenario from Task 2b is repeated using only $N = 10$ particles. This small number allows us to

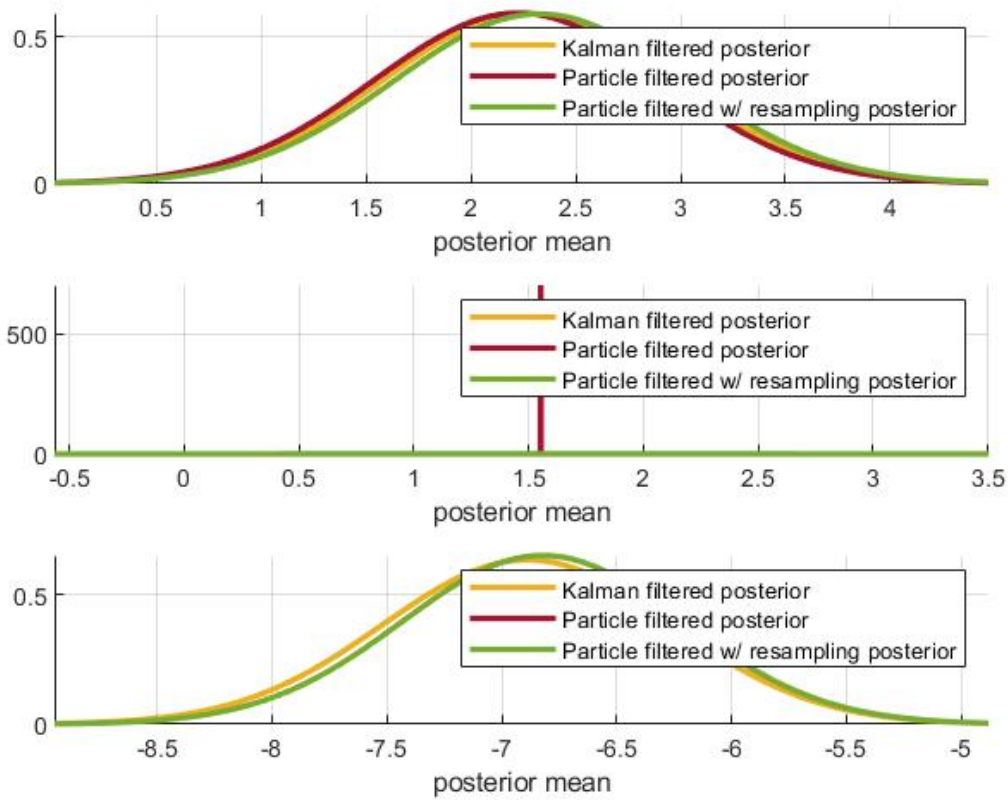


Figure 6: Posterior Approximations

trace the individual evolutionary paths of the particles.

Figure 8 vividly illustrates the failure of the un-resampled filter. The individual particle paths (faint red lines) originate at the incorrect prior near -20 and merely perform a random walk in that distant region. Because they are situated far from the true measurements (around 0 to 5), all particles immediately suffer from severe weight degeneracy. Lacking a mechanism to physically relocate the computational effort, the particles aimlessly drift, and the final state estimate remains entirely disconnected from reality.

Figure 9 demonstrates the powerful "survival of the fittest" mechanism introduced by resampling. Although the particles also start at -20 , process noise causes slight variations in their initial paths. As soon as a few particles drift marginally closer to the true measurements, they are assigned higher likelihood weights. The resampling algorithm aggressively duplicates these "fitter" particles while instantly eliminating the ones lingering at the bottom. This visually manifests as the particle bundle breaking away from the incorrect prior, aggressively migrating upward, and successfully enveloping the true trajectory by approximately $k = 12$.

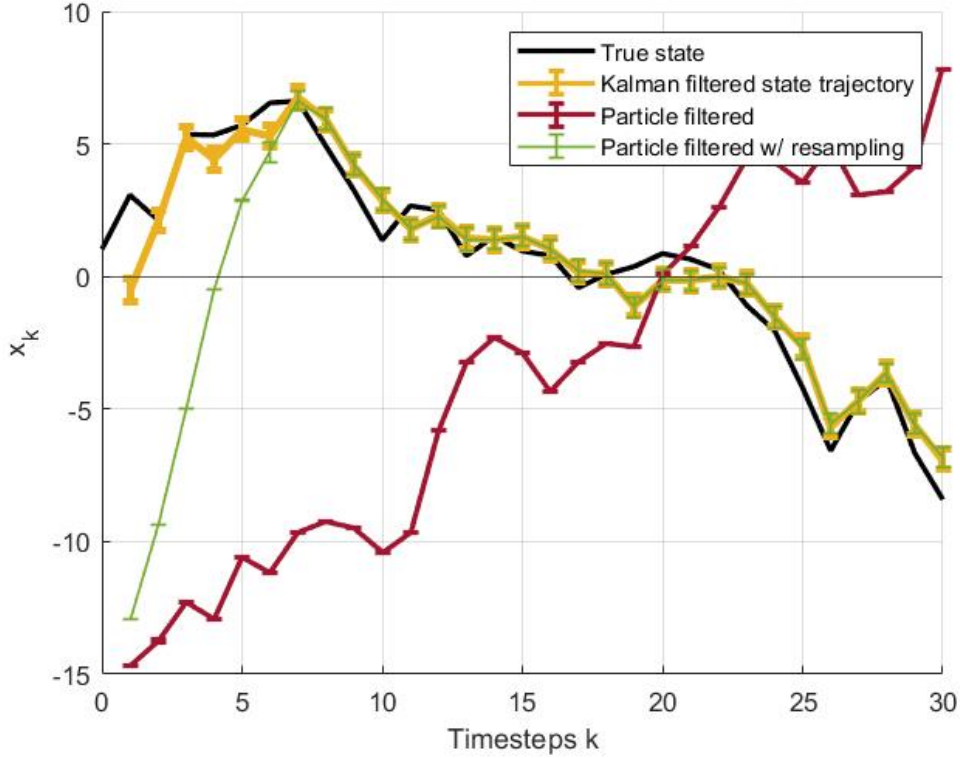


Figure 7: Catching up with incorrect prior

3 Bicycle tracking in a village

(a)

The bicycle trajectory was generated interactively using `MapProblemGetPoint.m`, by clicking along the roads of the village map. The resulting position sequence $\mathbf{x}_k^p$ is illustrated in Figure 10.

The trajectory was designed to resemble realistic bicycle movement: the bicycle starts in the upper-left area of the village, follows the left road southward, turns right along a horizontal road, continues through the central passage, and finally moves along the bottom road toward the right side of the village. The path follows the available roads and avoids all buildings, as required.

(b)

The velocity measurements are generated according to the measurement model.

$$\mathbf{y}_k = \mathbf{x}_k^v + \mathbf{r}_k, \quad (1)$$

where $\mathbf{x}_k^v = \mathbf{x}_k^p - \mathbf{x}_{k-1}^p$ is the true velocity at time step k , and $\mathbf{r}_k \sim \mathcal{N}(\mathbf{0}, \sigma_r^2 \mathbf{I}_{2 \times 2})$ is zero-mean Gaussian measurement noise with variance σ_r^2 in each dimension.

The measurement noise standard deviation was set to $\sigma_r = 0.1$, giving the measurement noise covariance.

$$\mathbf{R} = \sigma_r^2 \mathbf{I}_{2 \times 2} = 0.01 \cdot \mathbf{I}_{2 \times 2}. \quad (2)$$

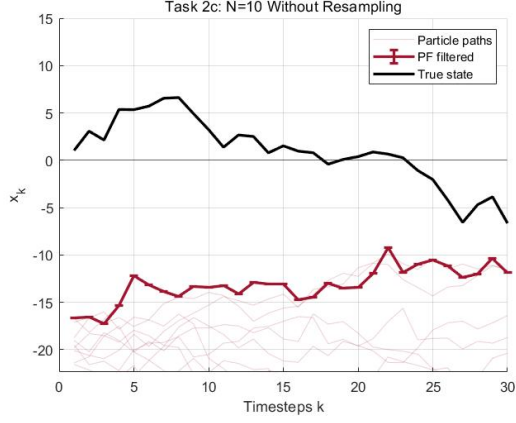


Figure 8: N=10 Without Resampling

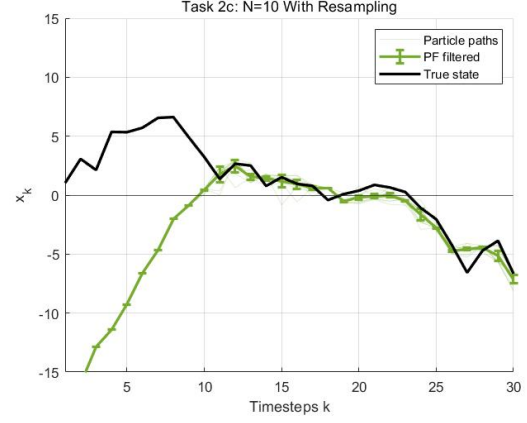


Figure 9: N=10 With Resampling

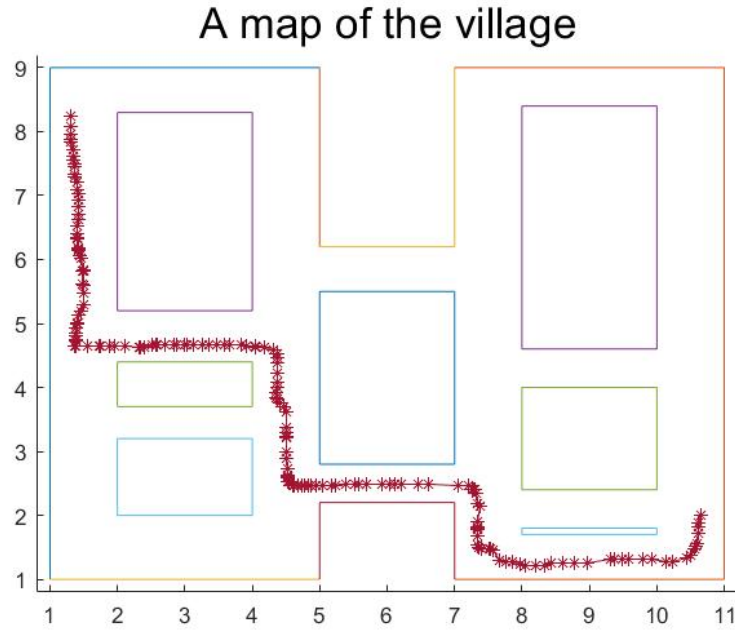


Figure 10: Bicycle trajectory

The velocity measurements $\mathbf{y}_k$ are two-dimensional, corresponding to the x - and y -components of the bicycle's displacement between consecutive positions. The measurements are corrupted by additive Gaussian noise, meaning that the observed velocity deviates slightly from the true velocity at each time step.

(c)

Based the constraint, both of as part of the measurement model or as part of the motion model can be incorporated into the filter. But we argue that it is more natural to treat it as a component of the motion model, for the following reasons.

Map as a measurement model

One could define a binary “measurement” that returns 1 if the position is on the road and 0 otherwise, effectively setting the likelihood to zero for particles inside buildings. However, this interpretation is problematic because no actual sensor produces this information — there is no physical measurement device that observes whether the bicycle is on the road. The map is prior knowledge about the environment, not an observation.

Map as a motion model

It is more natural to incorporate the map constraint directly into the motion model. In a particle filter, this is straightforward: after the prediction step, any particle whose predicted position falls inside a building is assigned a weight of zero, effectively rejecting trajectories that violate the map constraint. This can be written as

$$w_k^{(i)} \propto w_{k-1}^{(i)} \cdot p(\mathbf{y}_k \mid \mathbf{x}_k^{(i)}) \cdot \mathbf{1} \left[\text{isOnRoad} \left(\mathbf{x}_k^{(i)} \right) \right], \quad (3)$$

where $\mathbf{1}[\cdot]$ is the indicator function that equals 1 if the particle is on the road and 0 otherwise. This approach treats the map as a hard constraint on the state space, which is consistent with the physical interpretation that the bicycle *cannot* enter a building.

Why the particle filter is well suited

A key advantage of the particle filter in this context is that it can naturally handle this non-Gaussian, non-linear constraint. A Kalman-based filter would struggle to incorporate such a hard geometric constraint, since it assumes Gaussian distributions over the entire state space. The particle filter, on the other hand, can represent a multi-modal distribution — for example, when the bicycle is near a junction and could have turned in multiple directions — and simply discards particles that violate the map constraint by setting their weights to zero.

In summary, the map information is best viewed as a component of the motion model, implemented as a hard constraint that rejects particles in infeasible regions of the state space after each prediction step.

(d)

A particle filter was implemented to jointly track the position (x_k, y_k) and velocity $(v_{x,k}, v_{y,k})$ of the bicycle using a constant velocity motion model.

The initial position was assumed known, and $N = 2000$ particles were initialized around it with small random perturbations. At each time step, the map constraint was enforced by setting the weights of particles inside buildings to zero using `isOnRoad()`, and the particle weights were updated using the velocity measurements from Task 3b. Systematic resampling was applied at every time step to prevent particle degeneracy.

The process noise σ_q and measurement noise σ_r were turned empirically. Figure 11 shows the tracking results for different combinations of σ_q and σ_r .

From the results shown in Figure 11, the effects of σ_q and σ_r on filter performance can be clearly observed.

Increasing σ_q causes the particles to spread more widely during the prediction step, as seen when comparing Figure 11(a) and (d), while decreasing σ_q causes the particles to concentrate more tightly around the predicted position.

The measurement noise σ_r controls the sharpness of the likelihood function and thus the credibility assigned to each measurement. When σ_r is large, the likelihood becomes too flat to effectively distinguish between particles, causing the filter to rely almost entirely on the motion model and leading to particle divergence. Conversely, when σ_r is too small, the likelihood becomes overly sharp, and only very few particles receive non-negligible weights, which risks particle degeneracy.

In the limiting case where both $\sigma_q \rightarrow 0$ and $\sigma_r \rightarrow 0$, the particles collapse to a single point due to the absence of process noise, while simultaneously all particle weights become zero since no particle can exactly match the measurement. This causes the filter to completely break down, as illustrated in Figure 11(e).

Therefore, σ_q and σ_r must be balanced to achieve good tracking performance. Based on empirical tuning, the combination $\sigma_q = 0.05$ and $\sigma_r = 0.05$ was found to provide the best tracking result.

(e)

Figure 12 shows the final state of the particle filter with unknown initial position under two different measurement noise settings.

With $\sigma_r = 0.1$ (Figure 12(a)), the filter is able to partially position the bicycle. The particle cloud converges to the right side of the map near the true trajectory by the end of the sequence, and the estimated trajectory aligns reasonably well with the true trajectory in the later stages. However, the filter struggles in the early stages due to the lack of initial position information.

With $\sigma_r = 0.2$ (Figure 12(b)), the likelihood function is too flat to effectively distinguish between particles at different positions, so the particles remain spread across the entire road network throughout the sequence. The estimated trajectory is less stable and does not converge as clearly to the true trajectory.

The information enables localization

The filter localizes the bicycle through the combination of two sources of information.

1. Velocity measurements $\mathbf{y}_k$: At each time step, particles whose velocity state is inconsistent with the observed measurement receive a low likelihood weight and are gradually eliminated through resampling. This progressively rules out particles that are moving in the wrong direction or at the wrong speed.
2. Map constraints via `isOnRoad()`: After each prediction step, particles that drift into buildings are assigned zero weight. This is particularly powerful at narrow road segments and corners, where only particles on the correct road survive.

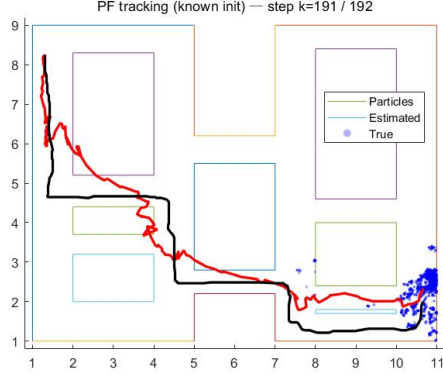
Events where large chunks of particles are ruled out

There are two types of events causing large numbers of particles to be eliminated simultaneously.

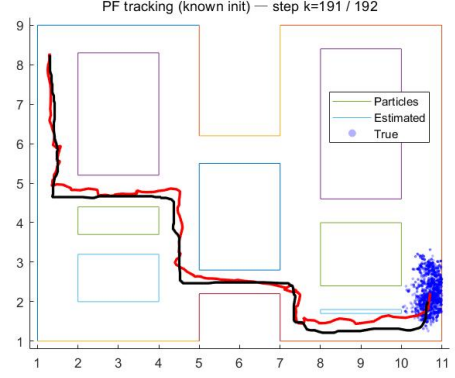
When the bicycle makes a 90-degree turn, particles that continue straight receive near-zero likelihood because their velocity direction is inconsistent with the measurement. This rapidly eliminates a large portion of the particle cloud, as can be seen in Figure 12(a) where particles are absent from the upper-left region of the map where the bicycle turned.

When the bicycle travels along a narrow road segment between buildings, the map constraint eliminates all particles that are not on that specific segment. This is visible in Figure 12(a) where the particle cloud is concentrated along the bottom road, consistent with the true trajectory.

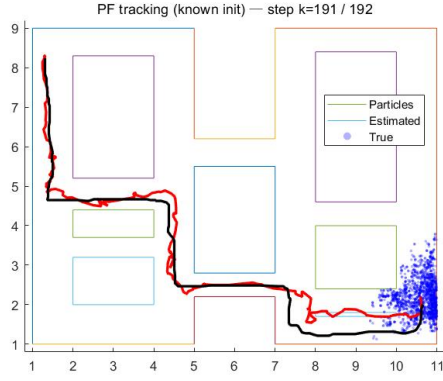
In conclusion, the filter with $\sigma_r = 0.1$ is able to position the bicycle, but only after sufficient measurements have been collected to distinguish the true trajectory from alternative hypotheses. The filter with $\sigma_r = 0.2$ fails to achieve reliable localization because the measurements are too noisy to provide meaningful weight updates.



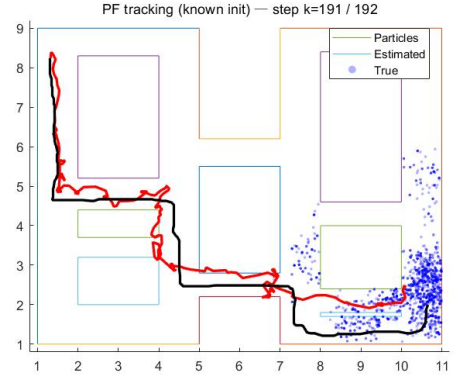
(a) $\sigma_q = 0.01, \sigma_r = 0.1$



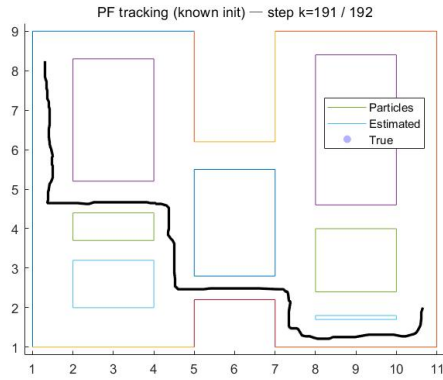
(b) $\sigma_q = 0.05, \sigma_r = 0.05$



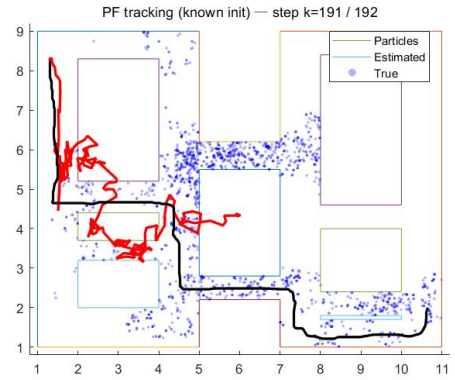
(c) $\sigma_q = 0.05, \sigma_r = 0.1$



(d) $\sigma_q = 0.15, \sigma_r = 0.1$

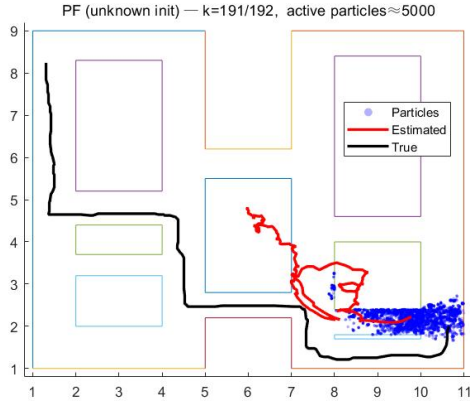


(e) $\sigma_q = 0, \sigma_r = 0$

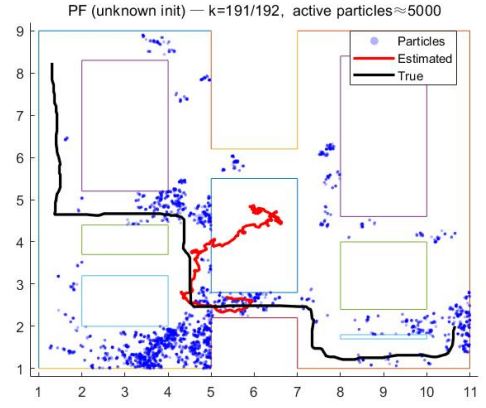


(f) $\sigma_q = 0.05, \sigma_r = 0.5$

Figure 11: Tracking results for different values of σ_q and σ_r



(a) $\sigma_q = 0.005$, $\sigma_r = 0.1$



(b) $\sigma_q = 0.005$, $\sigma_r = 0.2$

Figure 12: Final state of the particle filter with unknown initial position